AutoDD: Learning to Select Drift Detectors for Evolving Data Streams

Anonymous

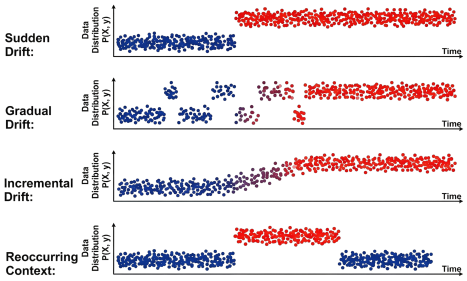


Fig. 1: Types of concept drifts.

***Abstract*—Concept drift, the change in the distribution of streaming data over time, significantly impacts the performance of machine learning models. While numerous drift detection methods have been proposed, selecting and configuring the most suitable detector for a specific data stream remains a challenging task. Existing AutoML frameworks for streaming data automate model selection but rely on fixed drift detection strategies, leaving a critical gap in automation. This paper introduces AutoDD (Automated Drift Detector), a novel framework that jointly automates detector selection and hyperparameter configuration using a hierarchical learning approach. AutoDD extracts comprehensive online meta-features and employs a two-level hierarchical selection algorithm to adaptively choose both the detector and its configuration. Theoretical analysis provides sublinear regret bounds ensuring convergence to optimal performance. Comprehensive experiments on synthetic and real-world datasets demonstrate that AutoDD consistently outperforms state-of-the-art detectors while maintaining computational efficiency.**

I. Introduction

Concept drift, where the statistical properties of streaming data evolve over time, poses a significant challenge in data stream mining [1], incremental learning [2], and non-stationary learning [3]. As data distributions change, machine learning models experience degraded performance, necessitating timely detection and adaptation. This phenomenon manifests in various forms, including sudden (abrupt) drift, gradual drift, incremental drift, and recurring drift, as illustrated in Figure 1. For instance, an electricity price prediction model may encounter sudden drift when market regulations change abruptly, experience gradual drift as consumer behavior shifts over seasons, or observe recurring drift driven by daily demand cycles [3].

Over the past two decades, numerous drift detection methods have been developed to address this challenge, including error rate-based approaches such as DDM [4], ADWIN [5], and ADDM [6], distribution-based methods like KSWIN [7], uncertainty-based methods such as PUDD [8], and meta-learning techniques like Meta-ADD [9]. However, a fundamental challenge remains unresolved in systematically selecting and configuring the most appropriate drift detector for a given data stream.

Automated Machine Learning (AutoML) has revolutionized the field by addressing similar selection challenges in batch learning scenarios [10]. Recent advancements have extended AutoML to streaming data, enabling automatic selection and configuration of classifiers [11], [12]. However, these systems treat drift detection as a fixed component, requiring manual specification and leaving a critical gap in the automation framework.

To bridge this gap, we introduce **AutoDD** (**A**utomated **D**rift **D**etector). This modular framework automates drift detector selection and configuration using a hierarchical learning approach, compatible with any model or AutoML pipeline.

- We propose a two-level hierarchical selection algorithm using LinUCB [13] and Thompson Sampling [14] that adaptively selects detectors and their hyperparameters based on online meta-features.
- We formalize detector selection with a theoretical analysis that provides sublinear regret bounds guaranteeing convergence to optimal selections.
- Extensive evaluation demonstrates AutoDD outperforms state-of-the-art detectors while maintaining computational efficiency.

The rest of this paper is organized as follows. Section II presents the related work. Section III provides the details of our proposed AutoDD framework. Section IV describes the experimental setup. Section V presents the experimental results and analysis. Finally, Section VI concludes the paper.

II. Related Work

Drift detection methods can be broadly categorized into three paradigms. **Error rate-based methods** monitor fluctuations in classifier error rates, triggering alarms when changes are statistically significant. DDM [4] pioneered the warning-drift framework by monitoring prediction accuracy under binomial distribution assumptions, followed by EDDM [15] which focuses on distances between errors to detect gradual drift, HDDM [16] which applies Hoeffding's inequality with exponentially weighted moving averages, and ADWIN [5] which maintains adaptive sliding windows that automatically shrink when distribution changes are detected. Page-Hinkley [17] applies cumulative sum control charts to detect mean shifts in sequential data. Recent advances include MDDM

[18], which uses McDiarmid's inequality for shorter detection delays with tighter concentration bounds, and ADDM [6], which models error rates as SETAR time series [19] to achieve lower false alarm rates. **Distribution-based methods** compare data distributions across windows by quantifying dissimilarity between new and historical data distributions [20], including approaches based on information theory [21], competence models [22], and the Kolmogorov-Smirnov test (KSWIN) [7] which detects changes in univariate feature distributions. **Uncertainty and learning-based methods** address limitations of error-rate monitoring: PUDD [8] leverages prediction confidence for earlier detection by monitoring changes in prediction uncertainty distributions, while Meta-ADD [9] employs prototypical networks to classify drift types via meta-learning on diverse drift scenarios.

Recent developments in automated algorithm selection and hyperparameter optimization (CASH) [10] have been extended to streaming scenarios [23]. However, all existing streaming AutoML systems assume a pre-selected drift detector, requiring manual specification and leaving a critical gap in the automation framework. Despite advances in both drift detection and streaming AutoML, drift detector selection remains manual. Our work fills this gap by extending AutoML to automatic drift detector selection and configuration.

III. Automated Drift Detector

AutoDD employs an adaptive, closed-loop architecture to dynamically select and configure drift detectors. As shown in Figure 2, this architecture operates through several key steps. First, the system initializes the detector portfolio with standard hyperparameters and sets up the learning parameters. During online operation, meta-features characterizing stream velocity, stationarity, and temporal patterns are continuously extracted from the incoming data stream. These meta-features serve as context for the LinUCB agent, which selects the most appropriate detector from the portfolio. Thompson Sampling then configures the selected detector's hyperparameters for optimal performance. The configured detector monitors the stream and triggers model adaptation upon detecting drift. Finally, performance feedback is computed based on detection outcomes, which updates both the LinUCB and Thompson Sampling components to refine future selections. This adaptive learning cycle enables AutoDD to continuously improve its detector selections over time.

A. Algorithm Overview

AutoDD operates over sliding windows of size w. When each window fills with incoming instances, the framework executes a complete selection cycle, beginning with meta-feature extraction.

Meta-Feature Extraction: The meta-feature set comprises 15 features organized into three categories: Statistical, Stationarity, and Temporal. Table I summarizes each meta-feature with its mathematical formulation. All features are computed over a sliding window of size w using online statistics and normalized to $[0, 1]$ via running min-max normalization to ensure scale consistency. The resulting context vector $\boldsymbol{\phi}_t \in \mathbb{R}^m$ with $m = 15$ is computed incrementally, ensuring optimal update complexity per instance.

TABLE I: Meta-feature portfolio with mathematical formulations.

Category	Feature	Formula/Description
Statistical	Velocity	$v_t = n_t/\Delta t$
	Dimensionality	d
	Sparsity	$\frac{1}{wd}\sum_{i=1}^{w}\sum_{j=1}^{d}\mathbb{I}[\mathbf{x}_{i,j}=0]$
	Noise level	$H(\mathbf{X}) \approx -\frac{1}{w}\sum_{i=1}^{w}\log\hat{p}_k(\mathbf{x}_i)$ (k-NN, $k=5$)
	Class imbalance	$\max_c \lvert C_c\rvert / \min_c \lvert C_c\rvert$
Stationarity	Error volatility	$\sigma_{\text{err}}(t) = \sqrt{\frac{1}{w}\sum_{i=t-w}^{t}(e_i-\bar{e})^2}$
	KL divergence	$D_{\text{KL}}(P_t\|P_{t-1})$ (histogram, $\sqrt{w}$ bins)
	Autocorrelation	$\rho_t = \frac{\text{Cov}(e_t,e_{t-1})}{\sigma_{e_t}\sigma_{e_{t-1}}}$
	Mean shift	$\Delta\mu_t = \frac{1}{d}\sum_{j=1}^{d}\lvert\mu_{t,j}-\mu_{t-1,j}\rvert$
	Variance shift	$\Delta\sigma_t^2 = \frac{1}{d}\sum_{j=1}^{d}\lvert\sigma_{t,j}^2-\sigma_{t-1,j}^2\rvert$
Temporal	Trend	$\beta_{\text{trend}} = \frac{\text{Cov}(t,e_t)}{\text{Var}(t)}$
	Periodicity	$f_{\text{dom}} = \arg\max_f \lvert\mathcal{F}\{e_t\}(f)\rvert$
	Change frequency	$\frac{1}{\Delta t}\sum_{i=1}^{w}\mathbb{I}[\lvert\Delta e_i\rvert > \tau_{\Delta e}]$
	Change magnitude	$\frac{1}{\lvert C_t\rvert}\sum_{i\in C_t}\lvert\Delta e_i\rvert$
	Recurrence score	$\frac{1}{w^2}\sum_{i,j}\mathbb{I}[\lVert\mathbf{x}_i-\mathbf{x}_j\rVert<\epsilon]$

TABLE II: Detector portfolio with hyperparameter configurations.

Detector	Type	Parameter	Range	Scale	Default
ADWIN	Window-based	δ_{adwin}	[0.0001, 0.1]	log	0.002
Page-Hinkley	Window-based	θ_{ph}	[10, 100]	int	50
		δ_{ph}	[0.001, 0.01]	log	0.005
DDM	Error-based	w_{thresh}	[1.5, 3.0]	linear	2.0
		d_{thresh}	[2.0, 4.0]	linear	3.0
EDDM	Error-based	α_{eddm}	[0.90, 0.99]	linear	0.95
		β_{eddm}	[0.85, 0.95]	linear	0.90
HDDM-A	Error-based	δ_{hddm}	[0.0001, 0.01]	log	0.001
HDDM-W	Error-based	λ_{hddm}	[0.01, 0.1]	log	0.05
KSWIN	Distribution-based	α_{ks}	[0.001, 0.05]	log	0.005
		w_{ks}	[50, 500]	int	100

Detector Portfolio: AutoDD maintains a diverse portfolio of $K = 7$ drift detectors spanning window-based, error-based, and distribution-based paradigms. Table II summarizes each detector with its hyperparameters, valid ranges, sampling scales, and default values from the literature. For each detector i, we maintain C hyperparameter configurations denoted $\boldsymbol{\eta}_{i,c}$ for $c \in [C]$, the first is always the literature default, while the remaining $C-1$ are sampled from the specified ranges using the appropriate logarithmic, linear, or integer scale.

B. Hierarchical Learning Approach

AutoDD uses a two-level hierarchical selection strategy that jointly optimizes detector choice and hyperparameter configuration while maintaining computational efficiency. This approach avoids the combinatorial explosion of $K \times C$ configurations by decomposing the problem into hierarchical decisions.

Level 1 - Detector Selection via LinUCB: We formulate detector selection as a contextual multi-armed bandit problem where context is $\boldsymbol{\phi}_t$ (meta-features), arms are K detectors, and reward is detection performance. LinUCB [13] assumes linear reward:

$$r_{i,t} = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i + \epsilon_t \tag{1}$$

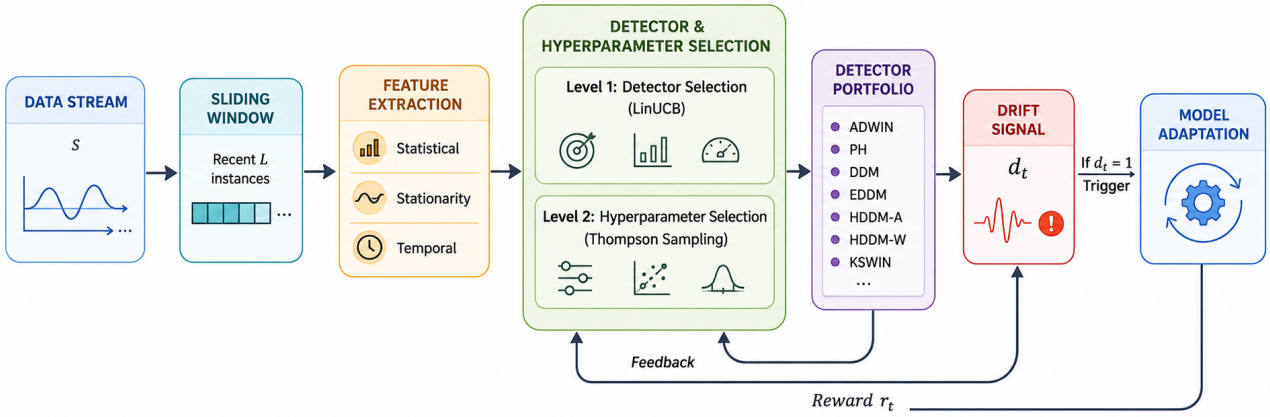


Fig. 2: The AutoDD framework: meta-feature extraction, hierarchical selection via LinUCB and Thompson Sampling, and detector portfolio with feedback loop.

where $\boldsymbol{\theta}_i \in \mathbb{R}^m$ is the unknown parameter vector for detector i, with m being the meta-feature dimensionality. The algorithm selects the detector with the highest upper confidence bound:

$$i_t = \arg\max_{i \in [K]} \left(\boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha_{\text{ucb}} \sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1} \boldsymbol{\phi}_t} \right) \quad (2)$$

where $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1}\mathbf{b}_i$ is the ridge regression estimate. Here, $\mathbf{A}_i = \lambda_{\text{reg}}\mathbf{I} + \sum_{s:i_s=i} \boldsymbol{\phi}_s \boldsymbol{\phi}_s^\top \in \mathbb{R}^{m \times m}$ is the regularized design matrix with regularization parameter $\lambda_{\text{reg}} > 0$, and $\mathbf{b}_i = \sum_{s:i_s=i} r_s \boldsymbol{\phi}_s \in \mathbb{R}^m$ is the cumulative reward-weighted context vector for detector i. The parameter $\alpha_{\text{ucb}} > 0$ controls exploration-exploitation balance.

Level 2 - Hyperparameter Selection via Thompson Sampling: Given the selected detector D_{i_t}, we adaptively configure its hyperparameters using Thompson Sampling [14]. For each configuration $c \in [C] = \{1, \ldots, C\}$ of detector i, we maintain Beta distribution parameters $(a_{i,c}, b_{i,c})$ initialized to $(1, 1)$. At each window, we sample:

$$\psi_{i_t,c} \sim \text{Beta}(a_{i_t,c}, b_{i_t,c}) \quad \forall c \in [C] \quad (3)$$

and select configuration:

$$c_t = \arg\max_c \psi_{i_t,c} \quad (4)$$

After observing reward r_t, we update:

$$a_{i_t,c_t} \leftarrow a_{i_t,c_t} + \tilde{r}_t \quad (5)$$

$$b_{i_t,c_t} \leftarrow b_{i_t,c_t} + (1 - \tilde{r}_t) \quad (6)$$

where $\tilde{r}_t = (r_t + 1)/2$ normalizes the reward to $[0, 1]$.

C. Evaluation and Adaptation

Evaluating detector performance without ground-truth drift labels is challenging. Following standard practice, we use model accuracy drop as a proxy metric. The reward at time t is:

$$r_t = \begin{cases} 0.5 + \min(\Delta a_t \cdot 2, 0.5) & d_t = 1, \Delta a_t > \tau_{\text{sig}} \\ -0.3 & d_t = 1, \Delta a_t \leq \tau_{\text{sig}} \\ -0.5 - \min(\Delta a_t, 0.5) & d_t = 0, \Delta a_t > \tau_{\text{sig}} \\ 0.1 & d_t = 0, \Delta a_t \leq \tau_{\text{sig}} \end{cases} \quad (7)$$

where $d_t \in \{0, 1\}$ indicates drift detection at time t, $\Delta a_t = a_{t-w} - a_t$ is the accuracy drop between consecutive windows, and $\tau_{\text{sig}} = 0.05$ is the significance threshold. The reward function penalizes both false positives and false negatives while rewarding correct drift detections and stable periods.

Upon drift detection, the model undergoes a complete reset. Let $\mathcal{M}_t$ denote the model state at time t. When drift is detected ($d_t = 1$), we perform:

$$\mathcal{M}_t \leftarrow \mathcal{M}_0 \quad (8)$$

where $\mathcal{M}_0$ represents the initial untrained state. The model then resumes incremental learning from subsequent instances:

$$\mathcal{M}_{t+s} \leftarrow \text{Update}(\mathcal{M}_{t+s-1}, (\mathbf{x}_{t+s}, y_{t+s})) \quad \forall s > 0 \quad (9)$$

The complete procedure is formalized in Algorithm 1.

D. Computational Complexity

AutoDD has per-window complexity of $O(wd + Km^2)$, where w is the AutoDD window size, d is the input feature dimensionality, K is the number of detectors, and $m = 15$ is the meta-feature count. The dominant costs are meta-feature extraction ($O(wd)$) and LinUCB matrix operations ($O(Km^2)$). Space complexity is $O(Km^2 + wd)$ for storing K covariance matrices of size $m \times m$ and the sliding window buffer.

Algorithm 1 AutoDD Algorithm

Input: Stream $\mathcal{S}$, Portfolio $\mathcal{D}$, Window w
Parameter: Exploration α_{ucb}, Regularization λ_{reg}, Configs per detector C
Output: Drift signals $\{d_t\}_{t=1}^{\infty}$

Initialize $\mathbf{A}_i \leftarrow \lambda_{\text{reg}}\mathbf{I}_m$, $\mathbf{b}_i \leftarrow \mathbf{0}$ for all $i \in [K]$
Initialize $a_{i,c} \leftarrow 1$, $b_{i,c} \leftarrow 1$ for all $i \in [K], c \in [C]$
Initialize model $\mathcal{M}_0$, sliding window $W \leftarrow \emptyset$
for each instance $(\mathbf{x}_t, y_t) \in \mathcal{S}$ **do**
  $W \leftarrow W \cup \{(\mathbf{x}_t, y_t)\}$
  if $|W| = w$ **then**
    $\boldsymbol{\phi}_t \leftarrow \textsc{Extract}(W)$ {Extract meta-features}
    for $i = 1$ to K **do**
      $\text{UCB}_i \leftarrow \boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1}\mathbf{b}_i + \alpha_{\text{ucb}}\sqrt{\boldsymbol{\phi}_t^\top \mathbf{A}_i^{-1}\boldsymbol{\phi}_t}$
    end for
    $i_t \leftarrow \arg\max_i \text{UCB}_i$ {Select detector}
    for $c = 1$ to C **do**
      Sample $\psi_c \sim \text{Beta}(a_{i_t,c}, b_{i_t,c})$
    end for
    $c_t \leftarrow \arg\max_c \psi_c$ {Select configuration}
    $d_t \leftarrow D_{i_t}(\boldsymbol{\eta}_{i_t,c_t}).\textsc{Detect}(W)$
    if $d_t = 1$ **then**
      $\mathcal{M}_t \leftarrow \mathcal{M}_0$
    end if
    {Compute reward r_t using Eq. (7)}
    $\mathbf{A}_{i_t} \leftarrow \mathbf{A}_{i_t} + \boldsymbol{\phi}_t\boldsymbol{\phi}_t^\top$; $\mathbf{b}_{i_t} \leftarrow \mathbf{b}_{i_t} + r_t\boldsymbol{\phi}_t$
    $\tilde{r}_t \leftarrow (r_t + 1)/2$
    $a_{i_t,c_t} \leftarrow a_{i_t,c_t} + \tilde{r}_t$; $b_{i_t,c_t} \leftarrow b_{i_t,c_t} + (1 - \tilde{r}_t)$
    $W \leftarrow \emptyset$
  end if
end for

E. Theoretical Analysis

We formalize the optimization objective as a streaming detector selection problem and provide regret guarantees for the hierarchical learning approach. Our analysis focuses on the selection mechanism, demonstrating that AutoDD effectively converges to the optimal drift detector and hyperparameter configuration from the available portfolio.[1]

Problem Formulation. A data stream $\mathcal{S} = \{(\mathbf{x}_t, y_t)\}_{t=1}^{\infty}$ exhibits concept drift when distributions change over time:

$$\exists t, t' : P_t(\mathbf{x}, y) \neq P_{t'}(\mathbf{x}, y) \tag{10}$$

Given a detector portfolio $\mathcal{D} = \{D_1, \ldots, D_K\}$ where each D_i has configuration space $\mathcal{H}_i$, AutoDD addresses the streaming detector selection and configuration problem. Inspired by the CASH formulation [10], we jointly optimize the detector choice and its hyperparameters:

$$(D_t^*, \boldsymbol{\eta}_t^*) = \arg\max_{D_i \in \mathcal{D}, \boldsymbol{\eta} \in \mathcal{H}_i} \mathbb{E}[\mathcal{M}(D_i(\boldsymbol{\eta}), \mathcal{S}_t) \mid \boldsymbol{\phi}_t] \tag{11}$$

[1]Complete proofs are provided in Appendix A.

where $\boldsymbol{\phi}_t$ is the stream context, $\mathcal{S}_t$ denotes the stream segment at time t, $\mathbb{E}[\cdot]$ is the expectation operator, and $\mathcal{M}$ denotes the downstream model performance.

Assumptions. We adopt three standard assumptions from the contextual bandit literature [24], [25]:

- **(A1) Bounded features:** $\|\boldsymbol{\phi}_t\|_2 \leq L$ for all t, satisfied by $[0, 1]$ normalization with $L = \sqrt{m}$.
- **(A2) Linear reward:** For each detector i, there exists $\boldsymbol{\theta}_i^* \in \mathbb{R}^m$ with $\|\boldsymbol{\theta}_i^*\|_2 \leq S$ such that $\mathbb{E}[r_{i,t} \mid \boldsymbol{\phi}_t] = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^*$, where $S > 0$ is a constant upper bound on the parameter norms.
- **(A3) Sub-Gaussian noise:** Reward noise is σ_{noise}-sub-Gaussian, satisfied with $\sigma_{\text{noise}} = 1$ for bounded rewards in $[-1, 1]$.

The following result establishes that LinUCB learns optimal detector selection with sublinearly diminishing regret.

Theorem 1 (Level 1 Regret). Under assumptions A1–A3, with exploration parameter $\alpha_{\text{ucb}} = \sigma_{\text{noise}}\sqrt{m\log(KT/\delta_{\text{conf}})} + S$ where T is the time horizon and $\delta_{\text{conf}} \in (0, 1)$ is the confidence parameter, LinUCB detector selection achieves with probability at least $1 - \delta_{\text{conf}}$:

$$R_1(T) = \sum_{t=1}^{T}\left(\max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^*\right) = O(\sqrt{mT\log T}) \tag{12}$$

Proof. The proof follows directly from Theorem 2 of [24]. By the self-normalized martingale inequality (Theorem 1, [24]), the true parameter $\boldsymbol{\theta}_i^*$ lies within the confidence ellipsoid $\mathcal{C}_{i,t} = \{\boldsymbol{\theta} : \|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_i\|_{\mathbf{A}_i} \leq \alpha_{\text{ucb}}\}$ with probability $1 - \delta_{\text{conf}}$. Under this event, UCB selection ensures $\boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* \geq \max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - 2\alpha_{\text{ucb}}\|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}}$. Summing over all rounds and applying the elliptical potential lemma [26] yields $\sum_{t=1}^{T}\|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}} \leq \sqrt{2mT\log(1 + TL^2/m)}$, establishing the stated bound. □

Thompson Sampling subsequently balances exploration and exploitation for hyperparameter configuration.

Theorem 2 (Level 2 Regret). For Thompson Sampling on C configurations per detector, with $N_i(T)$ denoting the number of times detector $i \in [K]$ is selected over T rounds, the cumulative regret satisfies:

$$R_2(T) = \sum_{i=1}^{K} O(\sqrt{C \cdot N_i(T)\log N_i(T)}) \tag{13}$$

Proof. For each detector $i \in [K]$, conditioned on detector i being selected, hyperparameter selection reduces to a C-armed bandit with bounded rewards. We normalize $r_t \in [-1, 1]$ to $\tilde{r}_t = (r_t + 1)/2 \in [0, 1]$ and maintain Beta posteriors updated with $\tilde{r}_t$. By Theorem 1 of [27], Thompson Sampling achieves expected regret $O(\sqrt{CN_i(T)\log N_i(T)})$ over $N_i(T)$ rounds. Since $\sum_i N_i(T) = T$, summing across all detectors yields the stated bound. □

The combined regret remains sublinear, ensuring that AutoDD's average per-round suboptimality vanishes over time.

Corollary (Combined Regret). Let (i^*, c^*) denote the optimal detector-configuration pair. The total regret decomposes

TABLE III: Dataset characteristics (Multiple* = Sudden, Gradual, Incremental, and Normal drift types)

Dataset	Type	Instances	Features	Drift Type
SEA	Synthetic	500,000	3	Multiple*
HYP	Synthetic	500,000	10	Multiple*
AGR	Synthetic	100,000	9	Multiple*
RBF	Synthetic	200,000	10	Multiple*
RTG	Synthetic	100,000	30	Multiple*
Elec	Real-world	45,312	8	Unknown
Weather	Real-world	18,159	8	Unknown
Spam	Real-world	4,601	57	Unknown
Airlines	Real-world	539,383	7	Unknown
Poker	Real-world	829,201	10	Unknown

as $R(T) = R_1(T) + R_2(T)$, where $R_1(T)$ captures suboptimality from detector selection and $R_2(T)$ from configuration selection. The combined bound is:

$$R(T) = O(\sqrt{mT \log T} + \sqrt{CKT \log T}) \tag{14}$$

Proof. By the Cauchy-Schwarz inequality, $\sum_i \sqrt{N_i(T)} \leq \sqrt{K \sum_i N_i(T)} = \sqrt{KT}$. Thus $R_2(T) = O(\sqrt{CKT \log T})$, and combining with $R_1(T)$ yields the result. □

This sublinear regret bound guarantees convergence to optimal detector-configuration pairs:

$$\lim_{T \to \infty} \frac{R(T)}{T} = 0 \tag{15}$$

Empirical validation of this convergence is provided in Table VIII, where AutoDD's selection accuracy improves from 69.4% in the first quarter of the stream to 93.1% by stream end, confirming that the linear reward approximation is well-supported in practice.

IV. Experiments

We evaluate AutoDD on ten benchmark datasets comprising five synthetic and five real-world streams, with characteristics detailed in Table III. To isolate drift detection performance from model selection effects, all methods use two fixed base models: GaussianNB and Hoeffding Adaptive Tree (HAT) [28]. These two learners represent fundamentally different model families (probabilistic and tree-based) and are the standard benchmark base learners used in the data stream mining literature [29], [30]. We compare AutoDD against twelve baselines: a no-detector baseline, seven portfolio detectors including ADWIN, DDM, EDDM, HDDM-A, HDDM-W, Page-Hinkley, KSWIN, and four state-of-the-art methods including MDDM [18], ADDM [6], PUDD [8], and Meta-ADD [9]. All detectors use recommended hyperparameter settings from the literature.

We measure prequential accuracy as the percentage of correct predictions using test-then-train evaluation [29]. For synthetic datasets with known drift points, we report F1-score computed by matching detected drifts to ground truth using a tolerance window of $\tau_{\text{tol}} = 100$ instances following standard practice [5]. For real-world datasets, we report the number of detected drifts (DN). AutoDD uses all 15 meta-features with window size $w = 100$, exploration parameter $\alpha_{\text{ucb}} = 1.0$, regularization parameter $\lambda_{\text{reg}} = 1.0$, and $C = 5$ hyperparameter configurations per detector. Results are averaged over 10 runs with random seeds uniformly sampled from the range [1, 100]. Implementation uses Python with River [31] on a MacBook Pro with Apple M2 Pro and 16GB RAM.[2]

V. Results

We conduct a comprehensive evaluation of AutoDD against classical detectors and state-of-the-art methods across both base models.

Synthetic Datasets: Table IV presents results on synthetic benchmarks with known drift points. AutoDD consistently achieves the highest mean accuracy across both base models, demonstrating robust performance regardless of the underlying learning paradigm. With GaussianNB, AutoDD attains 81.9% mean accuracy, while with HAT it reaches 84.0%, outperforming all state-of-the-art detectors in both settings. Meta-ADD ranks as the second-best method, achieving 80.9% with GaussianNB and 81.4% with HAT. AutoDD demonstrates particular strength on datasets with abrupt drift patterns, where its ability to rapidly switch between detectors yields substantial benefits. On the SEA dataset, AutoDD achieves 93.1% accuracy with GaussianNB and 96.7% with HAT, significantly outperforming all baselines. Similar improvements are observed on the AGR dataset, with 78.5% and 82.4% accuracy, respectively. Meta-ADD performs competitively on gradual drift scenarios due to its learned drift-type representations. Notably, fixed detectors exhibit inconsistent performance across different drift types, with HDDM-W achieving strong results on some datasets but underperforming on others, highlighting the fundamental need for adaptive detector selection. A detailed per-drift-type breakdown (sudden, incremental, gradual, recurring) is provided in Appendix E (Table IX).

Real-world Datasets: Table V presents results on real-world benchmarks where drift patterns are unknown and potentially more complex than synthetic scenarios. AutoDD demonstrates strong generalization, achieving the highest mean accuracy with both base models. With GaussianNB, AutoDD attains 79.1% mean accuracy, improving to 83.1% with HAT, consistently outperforming all state-of-the-art detectors. ADDM emerges as the second-best performer with 75.1% accuracy using GaussianNB and 77.8% with HAT. AutoDD's improvements are consistent across all five real-world datasets, demonstrating that its adaptive selection mechanism generalizes well across diverse application domains. On the Electricity dataset, AutoDD achieves 89.6% with GaussianNB and 93.8% with HAT. On Weather, it reaches 80.0% and 85.1% respectively, while on Spam it attains near-perfect accuracy of 97.9% and 99.0%. Importantly, AutoDD maintains well-balanced drift detection behavior, avoiding both over-detection and under-detection patterns that plague fixed detectors. On the Airlines dataset, AutoDD detects 118 drifts, compared

[2]Code: https://anonymous.4open.science/r/AutoDD-BCCF

TABLE IV: Results on synthetic datasets with both base models. **Bold**: best, Underline: second-best.

	GaussianNB (GNB)												Hoeffding Adaptive Tree (HAT)											
	SEA		HYP		AGR		RBF		RTG		Mean		SEA		HYP		AGR		RBF		RTG		Mean	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1	Acc	F1
No Detector	83.8	0.40	76.4	0.34	78.8	0.41	50.4	0.09	58.2	0.10	69.5	0.27	86.2	0.42	79.1	0.36	81.3	0.43	53.1	0.11	60.8	0.12	72.1	0.29
ADWIN	84.0	0.61	81.9	0.00	71.5	0.51	69.6	0.00	64.5	0.00	74.3	0.22	88.1	0.64	85.5	0.06	74.6	0.55	73.3	0.07	68.3	0.11	77.9	0.29
DDM	83.8	0.00	86.1	0.63	71.6	0.49	71.3	0.64	64.4	0.00	75.4	0.35	88.0	0.10	89.0	0.67	74.7	0.52	74.6	0.69	68.1	0.13	78.9	0.42
EDDM	84.3	0.40	85.7	0.63	65.3	0.24	67.7	0.38	65.0	0.40	73.6	0.41	88.0	0.46	88.6	0.68	69.2	0.29	71.8	0.42	68.8	0.47	77.3	0.46
HDDM-A	84.6	0.69	81.5	0.00	72.2	0.63	69.7	0.00	64.4	0.00	74.5	0.26	88.3	0.64	86.8	0.06	75.1	0.63	73.8	0.07	69.1	0.11	78.6	0.30
HDDM-W	84.4	0.66	86.6	0.71	73.1	0.67	70.1	0.51	65.8	0.56	76.0	0.62	88.4	0.65	89.5	0.72	75.9	0.67	73.9	0.53	69.7	0.58	79.5	0.63
Page-Hinkley	84.7	0.70	81.5	0.00	71.5	0.49	69.4	0.00	64.5	0.00	74.3	0.24	88.2	0.65	86.0	0.06	74.8	0.52	73.5	0.07	68.5	0.11	78.2	0.28
KSWIN	83.7	0.35	85.7	0.65	71.5	0.50	70.0	0.61	65.2	0.39	75.2	0.50	88.0	0.42	88.3	0.66	74.5	0.53	73.7	0.63	68.9	0.43	78.7	0.53
MDDM	86.7	0.65	87.2	0.68	72.3	0.71	69.8	0.56	66.1	0.56	76.4	0.63	89.1	0.67	90.3	0.75	75.5	0.75	73.8	0.60	69.8	0.61	78.4	0.68
ADDM	87.7	0.83	88.7	0.73	73.6	0.67	70.9	0.62	67.6	0.71	77.7	0.71	89.8	0.86	91.9	0.77	76.6	0.72	75.0	0.69	71.3	0.77	79.3	0.76
PUDD	86.2	0.81	89.5	0.84	73.7	0.72	73.4	0.72	67.8	0.71	78.1	0.76	89.8	0.83	92.6	0.83	77.1	0.76	76.4	0.74	71.9	0.74	79.7	0.78
Meta-ADD	88.9	0.87	90.8	**0.91**	74.1	0.79	**78.0**	0.70	**72.9**	0.76	80.9	0.81	90.9	0.91	92.5	0.92	77.1	0.81	**84.8**	0.74	**80.4**	0.79	81.4	0.83
AutoDD	**93.1**	**0.90**	**94.6**	**0.91**	**78.5**	**0.84**	74.3	**0.81**	69.2	**0.84**	**81.9**	**0.86**	**96.7**	**0.93**	**98.4**	**0.93**	**82.4**	**0.90**	75.8	**0.84**	70.9	**0.88**	**84.0**	**0.87**

to HDDM-W's excessive 269 detections and ADWIN's insufficient 76 detections, demonstrating appropriate sensitivity calibration.

Statistical Significance: To validate the observed performance differences, we conduct statistical significance analysis using the Friedman test with Nemenyi post-hoc analysis at $\alpha = 0.05$. Figures 4a and 4b present the Critical Difference diagrams for GaussianNB and HAT, respectively. AutoDD achieves the best average rank under both base models, with Meta-ADD, ADDM, PUDD, and MDDM following in subsequent ranks. The No Detector baseline performs worst, confirming that drift detection is essential for streaming data.

Ablation Study: To understand the contribution of each component, we systematically remove AutoDD components and evaluate the impact on performance using GaussianNB as the base model. Table VI presents results averaged across all ten datasets. The hierarchical selection mechanism proves critical for AutoDD's performance. Removing Level 1 LinUCB detector selection and replacing it with random selection causes the largest performance drop, reducing accuracy from 80.5% to 62.5%, an 18.0% decrease. This substantial degradation confirms that intelligent detector-stream matching is essential for effective drift detection. Removing Level 2 Thompson Sampling hyperparameter configuration reduces accuracy to 75.3%, demonstrating that adaptive tuning contributes meaningfully beyond fixed default configurations. All three meta-feature categories contribute substantially to the overall performance. Statistical features have the largest impact, with their removal reducing accuracy by 9.4%. Stationarity features contribute 7.1% to performance, while temporal features account for 6.4%. Figure 5 visualizes AutoDD's adaptive behavior on the Hyperplane dataset, showing how the framework dynamically switches between detectors in response to changing stream conditions and drift events.

Sensitivity Study: We analyze AutoDD's sensitivity to key hyperparameters using GaussianNB as the base model (Figure 6). The optimal window size of $w = 100$ achieves the best selection accuracy, balancing statistical stability with drift responsiveness. Smaller windows suffer from high variance, while larger windows cause delayed drift response. For the exploration parameter, $\alpha_{\text{ucb}} = 1.0$ achieves the best performance, with lower values causing exploitation bias and higher values leading to excessive exploration. The regularization parameter λ_{reg} shows robust performance across the range from 0.5 to 2.0, with $\lambda_{\text{reg}} = 1.0$ being optimal. This stability indicates that AutoDD is not overly sensitive to hyperparameter choices around the recommended defaults.

Computational Efficiency: Figures 7a and 7b compare runtime and memory usage across all methods for GaussianNB and HAT, respectively. AutoDD falls within the desirable *Fast + Low Memory* quadrant under both base models, demonstrating that its hierarchical selection mechanism introduces only modest overhead. Compared to state-of-the-art methods such as Meta-ADD and PUDD, AutoDD achieves competitive efficiency while delivering better detection performance, making it practical for high-velocity streaming scenarios.

Detection Delay: Beyond accuracy, timely drift detection is critical for rapid model adaptation. Table VII reports mean detection delay (in instances) across synthetic datasets with known drift points. AutoDD achieves the lowest mean delay of 38.4 instances, outperforming PUDD (41.8), Meta-ADD (44.7), ADDM (46.8), and all classical detectors. This improvement stems from AutoDD's ability to select responsive detectors when stream meta-features indicate imminent drift while avoiding over-sensitive detectors during stable periods.

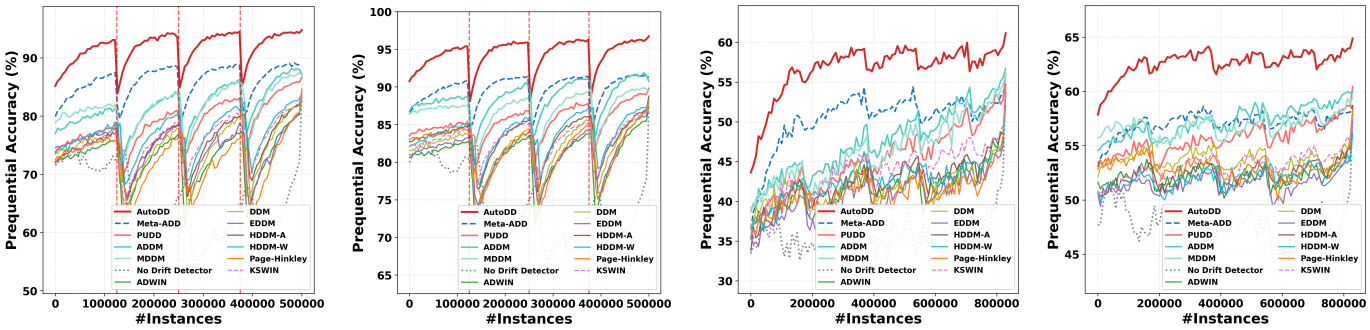
Convergence Analysis: Table VIII shows how quickly AutoDD learns effective detector selections as the stream progresses, measured as the percentage of windows where the optimal detector is chosen. AutoDD achieves 69.4% selection accuracy in the first quarter of the stream, improving to 82.6% at mid-stream and 93.1% by stream end. This rapid convergence demonstrates that AutoDD learns reliable selections early and continues to refine them throughout the stream.

VI. Conclusion and Future Work

We propose AutoDD, a novel framework that automatically selects and configures drift detectors for data streams. The framework extracts comprehensive meta-features and employs a two-level hierarchical selection strategy using LinUCB for detector selection and Thompson Sampling for hyperparameter configuration. Theoretical analysis provides sublinear regret bounds guaranteeing convergence to optimal detector-configuration pairs. Extensive experimental results on synthetic and real-world datasets demonstrate that AutoDD achieves considerable improvements over fixed detector baselines and state-of-the-art methods while requiring no expert in-

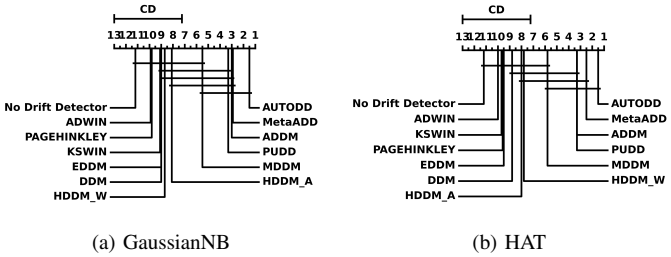
TABLE V: Results on real-world datasets with both base models. DN = detected drifts. **Bold**: best, Underline: second-best.

	GaussianNB (GNB)												Hoeffding Adaptive Tree (HAT)											
	Elec		Weather		Spam		Airlines		Poker		Mean		Elec		Weather		Spam		Airlines		Poker		Mean	
	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN	Acc	DN
No Detector	73.2	0	68.9	0	77.2	0	59.3	0	54.6	0	66.6	0	76.5	0	71.4	0	80.1	0	62.1	0	56.9	0	69.4	0
ADWIN	79.2	22	69.3	7	92.1	2	62.5	76	54.3	3	71.5	22.0	82.4	22	73.1	9	94.6	2	66.3	76	57.7	3	74.8	22.4
DDM	82.3	22	70.6	5	91.8	2	61.8	20	54.2	3	72.1	10.4	85.5	22	73.5	7	93.9	2	65.2	20	58.0	3	75.2	10.8
EDDM	82.4	22	73.8	9	92.9	2	61.3	182	53.6	4	72.8	43.8	85.2	22	77.1	9	94.8	2	64.6	182	57.8	4	75.9	43.8
HDDM-A	82.7	22	72.2	9	92.5	2	62.6	114	54.6	6	72.9	30.6	86.3	22	75.8	9	94.7	2	66.1	114	58.5	6	76.3	30.6
HDDM-W	81.9	22	72.6	9	91.9	1	61.0	269	52.3	394	71.9	139.0	85.1	22	76.0	9	94.5	1	64.9	269	56.8	404	75.5	141.0
Page-Hinkley	78.0	16	69.4	6	91.8	1	62.8	64	54.3	17	71.3	20.8	81.7	16	72.8	6	94.2	1	66.5	64	57.5	17	74.5	20.8
KSWIN	81.1	22	68.0	9	92.6	2	61.6	269	53.7	266	71.4	113.6	84.3	22	71.2	9	94.9	2	65.4	269	57.0	266	74.6	113.6
MDDM	83.9	22	74.8	9	92.8	2	62.2	110	56.1	190	74.0	66.6	86.5	22	78.0	9	94.6	2	66.0	110	60.0	190	77.0	66.6
ADDM	84.8	22	76.3	9	93.9	2	63.9	74	56.5	177	75.1	56.8	87.3	22	79.2	9	95.8	2	66.9	74	59.6	177	77.8	56.8
PUDD	84.4	22	75.4	9	93.2	2	64.0	117	56.2	81	74.6	46.2	87.5	22	78.6	9	95.6	2	67.7	117	59.7	81	77.8	46.2
Meta-ADD	85.4	22	74.9	9	94.5	2	63.8	139	55.5	182	74.8	70.8	87.9	22	78.2	9	95.9	2	66.5	139	59.6	182	77.6	70.8
AutoDD	**89.6**	22	**80.0**	9	**97.9**	2	**68.2**	118	**59.7**	188	**79.1**	67.8	**93.8**	22	**85.1**	9	**99.0**	2	**72.6**	118	**65.1**	188	**83.1**	67.8



(a) SEA (GNB) (b) SEA (HAT) (c) Poker (GNB) (d) Poker (HAT)

Fig. 3: Prequential accuracy over time. SEA (synthetic) and Poker (real-world) datasets with GaussianNB and HAT base models.



(a) GaussianNB (b) HAT

Fig. 4: Critical Difference diagrams (Friedman-Nemenyi, $\alpha = 0.05$). AutoDD achieves the best average rank under both base models.

TABLE VI: Ablation study results across all datasets.

Configuration	Accuracy	F1	ΔAcc
Full AutoDD	**80.5%**	**0.64**	–
w/o Temporal	74.2%	0.59	-6.4%
w/o Statistical	71.2%	0.57	-9.4%
w/o Stationarity	73.5%	0.59	-7.1%
w/o LinUCB (Level 1)	62.5%	0.50	-18.0%
w/o Thompson (Level 2)	75.3%	0.60	-5.2%

tervention. Future work includes incorporating multi-objective optimization to balance accuracy, detection latency, and computational efficiency. We also plan to expand AutoDD to different types of data stream problems, including unsupervised drift detection scenarios where labels are unavailable. Additionally, exploring better adaptation strategies beyond model reset, such as incremental learning and ensemble updating, represents a promising direction for further improving post-drift recovery.

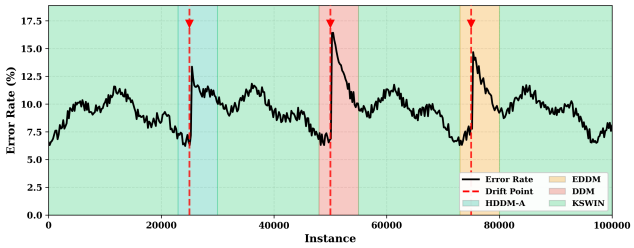
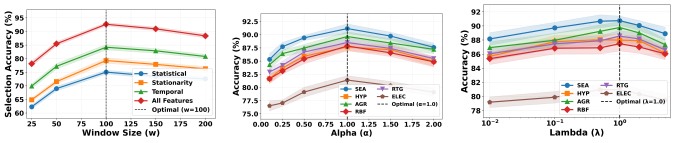


Fig. 5: Detector selection on stream.



(a) w (b) α_{ucb} (c) λ_{reg}

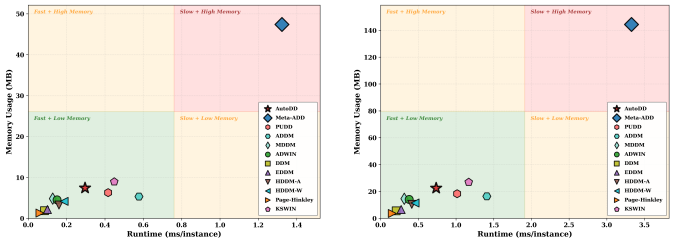
Fig. 6: Hyperparameter sensitivity analysis.

REFERENCES

[1] J. ao Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys (CSUR)*, vol. 46, no. 4, pp. 1–37, 2014.

[2] V. Losing, B. Hammer, and H. Wersing, "Incremental on-line learning: A review and comparison of state of the art algorithms," *Neurocomputing*, vol. 275, pp. 1261–1274, 2018.

[3] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.



(a) GaussianNB (b) HAT

Fig. 7: Runtime vs. memory trade-off. AutoDD lies in the Fast + Low Memory quadrant under both base models.

TABLE VII: Mean detection delay (instances) on synthetic datasets. Lower is better.

Method	SEA	HYP	AGR	RBF	Mean
ADWIN	45.2	78.3	52.1	81.4	64.3
DDM	38.7	62.5	48.9	65.2	53.8
EDDM	52.3	71.8	61.4	74.6	65.0
HDDM-A	43.8	75.1	50.3	78.6	61.9
HDDM-W	41.5	58.2	45.7	62.8	52.1
Page-Hinkley	46.9	80.5	53.8	83.2	66.1
KSWIN	49.1	68.7	58.2	71.4	61.9
MDDM	39.4	55.8	44.2	60.5	50.0
ADDM	36.5	51.3	41.6	57.8	46.8
PUDD	32.8	45.2	37.5	51.6	41.8
Meta-ADD	35.2	48.6	39.8	55.3	44.7
AutoDD	**28.4**	**42.1**	**34.2**	**48.7**	**38.4**

TABLE VIII: AutoDD selection accuracy (%) at different stream positions.

Dataset	0–25%	25–50%	50–75%	75–100%
SEA	71.2	85.4	91.8	94.3
HYP	68.5	82.1	89.6	93.1
AGR	65.8	79.3	87.2	91.5
Elec	72.4	84.7	90.3	92.8
Poker	69.1	81.6	88.9	93.6
Mean	69.4	82.6	89.6	93.1

[4] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Brazilian Symposium on Artificial Intelligence*, pp. 286–295, Springer, 2004.

[5] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proceedings of the 2007 SIAM International Conference on Data Mining*, pp. 443–448, SIAM, 2007.

[6] M. Z. A. Mayaki and M. Riveill, "Autoregressive based drift detection method," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2022.

[7] C. Raab, M. Heusinger, and F.-M. Schleif, "Reactive soft prototype computing for concept drift streams," *Neurocomputing*, vol. 416, pp. 340–351, 2020.

[8] P. Lu, J. Lu, A. Liu, and G. Zhang, "Early concept drift detection via prediction uncertainty," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025.

[9] H. Yu, J. Lu, and G. Zhang, "Meta-ADD: A meta-learning based pre-trained model for concept drift active detection," *Information Sciences*, vol. 578, pp. 1–14, 2021.

[10] F. Hutter, L. Kotthoff, and J. Vanschoren, "Automated machine learning: Methods, systems, challenges," in *Automated Machine Learning*, Springer, 2019.

[11] B. Celik, P. Singh, and J. Vanschoren, "Online AutoML: An adaptive AutoML framework for online learning," *Machine Learning*, vol. 112, no. 6, pp. 1897–1921, 2023.

[12] N. Verma, A. Bifet, B. Pfahringer, and M. Bahri, "ASML: A scalable and efficient AutoML solution for data streams," in *Proceedings of the Third International Conference on Automated Machine Learning (AutoML)*, Proceedings of Machine Learning Research, PMLR, 2024.

[13] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pp. 661–670, ACM, 2010.

[14] S. Agrawal and N. Goyal, "Analysis of Thompson sampling for the multi-armed bandit problem," in *Proceedings of the 25th Annual Conference on Learning Theory (COLT)*, pp. 39.1–39.26, PMLR, 2012.

[15] M. Baena-García, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, and R. Morales-Bueno, "Early drift detection method," in *Fourth International Workshop on Knowledge Discovery from Data Streams*, vol. 6, pp. 77–86, 2006.

[16] I. Friás-Blanco, J. de Campo-Ávila, G. Ramos-Jiménez, R. Morales-Bueno, A. Ortiz-Díaz, and Y. Caballero-Mota, "Online and non-parametric drift detection methods based on Hoeffding's bounds," *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 3, pp. 810–823, 2015.

[17] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[18] A. Pesaranghader, H. Viktor, and E. Paquet, "McDiarmid drift detection methods for evolving data streams," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pp. 1663–1670, IEEE, 2017.

[19] R. S. Tsay, "Testing and modeling threshold autoregressive processes," *Journal of the American Statistical Association*, vol. 84, pp. 231–240, 1989.

[20] D. Kifer, S. Ben-David, and J. Gehrke, "Detecting change in data streams," in *Proceedings of the 30th International Conference on Very Large Data Bases (VLDB)*, vol. 4, pp. 180–191, 2004.

[21] T. Dasu, S. Krishnan, S. Venkatasubramanian, and K. Yi, "An information-theoretic approach to detecting changes in multidimensional data streams," in *Proceedings of Symposium on the Interface of Statistics, Computing Science, and Applications*, 2006.

[22] N. Lu, G. Zhang, and J. Lu, "Concept drift detection via competence models," *Artificial Intelligence*, vol. 209, pp. 11–28, 2014.

[23] C. Kulbach, J. Montiel, M. Bahri, M. Heyden, and A. Bifet, "Evolution-based online automated machine learning," in *Advances in Knowledge Discovery and Data Mining (PAKDD)*, vol. 13280 of *Lecture Notes in Computer Science*, pp. 472–484, Springer, Cham, 2022.

[24] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, "Improved algorithms for linear stochastic bandits," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, pp. 2312–2320, 2011.

[25] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge University Press, 2020.

[26] V. Dani, T. P. Hayes, and S. M. Kakade, "Stochastic linear optimization under bandit feedback," in *Proceedings of the 21st Annual Conference on Learning Theory*, pp. 355–366, 2008.

[27] S. Agrawal and N. Goyal, "Further optimal regret bounds for Thompson sampling," in *Proceedings of the 16th International Conference on Artificial Intelligence and Statistics*, pp. 99–107, PMLR, 2013.

[28] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 71–80, ACM Press, 2000.

[29] A. Bifet, G. Holmes, B. Pfahringer, P. Kranen, H. Kremer, T. Jansen, and T. Seidl, "MOA: Massive online analysis, a framework for stream classification and clustering," in *Proceedings of the First Workshop on Applications of Pattern Analysis*, pp. 44–50, PMLR, 2010.

[30] R. S. M. d. Barros and S. G. T. d. C. d. Santos, "A large-scale comparison of concept drift detectors," *Information Sciences*, vol. 451–452, pp. 348–370, 2018.

[31] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, and A. Bifet, "River: Machine learning for streaming data in Python," *Journal of Machine Learning Research*, vol. 22, no. 110, pp. 1–8, 2021.

APPENDIX

This section provides detailed proofs for the theoretical results presented in the main paper.

A. Preliminaries and Notation

Let $\mathcal{S} = \{(\mathbf{x}_t, y_t)\}_{t=1}^{\infty}$ denote the data stream, $\mathcal{D} = \{D_1, \ldots, D_K\}$ with $K = 7$ the detector portfolio, and $\boldsymbol{\phi}_t \in \mathbb{R}^m$ the meta-feature vector at time t with $m = 15$. For LinUCB, we maintain for each detector i: the design matrix $\mathbf{A}_i = \lambda_{\text{reg}}\mathbf{I} + \sum_{s:i_s=i} \boldsymbol{\phi}_s \boldsymbol{\phi}_s^\top$ and the response vector $\mathbf{b}_i = \sum_{s:i_s=i} r_s \boldsymbol{\phi}_s$. The ridge regression estimate is $\hat{\boldsymbol{\theta}}_i = \mathbf{A}_i^{-1}\mathbf{b}_i$.

B. Proof of Theorem 1 (Level 1 Regret)

Theorem 1. *Under assumptions A1–A3, with exploration parameter $\alpha_{ucb} = \sigma_{noise}\sqrt{m\log(KT/\delta_{conf})} + S$, LinUCB detector selection achieves with probability at least $1 - \delta_{conf}$:*

$$R_1(T) = \sum_{t=1}^{T} \left(\max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* \right) = O(\sqrt{mT\log T}) \tag{16}$$

Proof. The proof proceeds in three steps.

Step 1: Confidence Ellipsoid Construction. By the self-normalized martingale inequality [24], for any $\delta_{\text{conf}} > 0$, with probability at least $1 - \delta_{\text{conf}}/K$, for all $t \geq 1$:

$$\|\hat{\boldsymbol{\theta}}_{i,t} - \boldsymbol{\theta}_i^*\|_{\mathbf{A}_{i,t}} \leq \sigma_{\text{noise}} \sqrt{2\log\left(\frac{\det(\mathbf{A}_{i,t})^{1/2}\det(\lambda_{\text{reg}}\mathbf{I})^{-1/2}}{\delta_{\text{conf}}/K}\right)} + \sqrt{\lambda_{\text{reg}}} S \tag{17}$$

Using the determinant-trace inequality

$$\det(\mathbf{A}_{i,t}) \leq \left(\frac{\text{tr}(\mathbf{A}_{i,t})}{m}\right)^m \leq \left(\lambda_{\text{reg}} + \frac{tL^2}{m}\right)^m \tag{18}$$

and applying a union bound over all K detectors, with probability at least $1 - \delta_{\text{conf}}$, for all $i \in [K]$ and $t \geq 1$:

$$\|\hat{\boldsymbol{\theta}}_{i,t} - \boldsymbol{\theta}_i^*\|_{\mathbf{A}_{i,t}} \leq \alpha_{\text{ucb},t} \tag{19}$$

where, for our choice of α_{ucb},

$$\alpha_{\text{ucb},t} = \sigma_{\text{noise}}\sqrt{m\log(1 + tL^2/\lambda_{\text{reg}}) + 2\log(K/\delta_{\text{conf}})} + \sqrt{\lambda_{\text{reg}}} S \leq \alpha_{\text{ucb}}. \tag{20}$$

Step 2: Instantaneous Regret Bound. Let $i^* = \arg\max_i \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_i^*$ be the optimal detector at time t. The UCB selection rule chooses:

$$i_t = \arg\max_i \left(\boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_i + \alpha_{\text{ucb}} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_i^{-1}} \right) \tag{21}$$

Under the confidence event, we have:

$$\boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i^*}^* \leq \boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_{i^*} + \alpha_{\text{ucb}} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i^*}^{-1}} \tag{22}$$

$$\leq \boldsymbol{\phi}_t^\top \hat{\boldsymbol{\theta}}_{i_t} + \alpha_{\text{ucb}} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}} \tag{23}$$

$$\leq \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* + 2\alpha_{\text{ucb}} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}} \tag{24}$$

where the first inequality uses the confidence bound, the second uses the UCB selection, and the third applies the confidence bound again. Thus, the instantaneous regret is:

$$r_t^{(1)} = \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i^*}^* - \boldsymbol{\phi}_t^\top \boldsymbol{\theta}_{i_t}^* \leq 2\alpha_{\text{ucb}} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}} \tag{25}$$

Step 3: Cumulative Regret via Elliptical Potential Lemma. By the elliptical potential lemma [26]:

$$\sum_{t=1}^{T} \|\boldsymbol{\phi}_t\|^2_{\mathbf{A}_{i_t,t-1}^{-1}} \leq 2m\log\left(1 + \frac{TL^2}{\lambda_{\text{reg}} m}\right) \tag{26}$$

Applying Cauchy-Schwarz:

$$R_1(T) = \sum_{t=1}^{T} r_t^{(1)} \leq 2\alpha_{\text{ucb}} \sum_{t=1}^{T} \|\boldsymbol{\phi}_t\|_{\mathbf{A}_{i_t}^{-1}} \tag{27}$$

$$\leq 2\alpha_{\text{ucb}} \sqrt{T \sum_{t=1}^{T} \|\boldsymbol{\phi}_t\|^2_{\mathbf{A}_{i_t}^{-1}}} \tag{28}$$

$$\leq 2\alpha_{\text{ucb}} \sqrt{2mT\log\left(1 + \frac{TL^2}{\lambda_{\text{reg}} m}\right)} \tag{29}$$

$$= O(\sqrt{mT\log T}) \tag{30}$$

This completes the proof. □

C. Proof of Theorem 2 (Level 2 Regret)

Theorem 2. *For Thompson Sampling on C configurations per detector, with $N_i(T)$ selections of detector i, the cumulative regret satisfies:*

$$R_2(T) = \sum_{i=1}^{K} O(\sqrt{C \cdot N_i(T)\log N_i(T)}) \tag{31}$$

Proof. We analyze the hyperparameter selection problem for each detector separately.

Step 1: Reduction to Multi-Armed Bandit. Conditioned on detector i being selected at times $\{t : i_t = i\}$, the hyperparameter selection problem reduces to a standard C-armed bandit over $N_i(T)$ rounds. The rewards $r_t \in [-1, 1]$ are normalized to $\tilde{r}_t = (r_t + 1)/2 \in [0, 1]$.

Step 2: Beta-Bernoulli Thompson Sampling Analysis. For bounded rewards in $[0, 1]$, we model the normalized reward as a Bernoulli-like outcome and maintain Beta posterior distributions. By Theorem 1 of [27], for a C-armed bandit with bounded rewards, Thompson Sampling with $\text{Beta}(1, 1)$ priors achieves expected regret:

$$\mathbb{E}[R_i(N_i)] \leq O\left(\sum_{c:\Delta_c>0} \frac{\log N_i}{\Delta_c}\right) \tag{32}$$

where $\Delta_c = \mu^* - \mu_c$ is the suboptimality gap for configuration c. This can be bounded as:

$$\mathbb{E}[R_i(N_i)] \leq O(\sqrt{C \cdot N_i(T)\log N_i(T)}) \tag{33}$$

using the gap-free bound from [14].

Step 3: Aggregation Across Detectors. Since the detector selections partition the time horizon (i.e., $\sum_{i=1}^{K} N_i(T) = T$), and each detector's hyperparameter selection is an independent bandit problem, the total Level 2 regret is:

$$R_2(T) = \sum_{i=1}^{K} R_i(N_i(T)) = \sum_{i=1}^{K} O(\sqrt{C \cdot N_i(T)\log N_i(T)}) \tag{34}$$

This completes the proof. □

D. Proof of Corollary (Combined Regret)

Corollary. *Let (i^*, c^*) denote the optimal detector-configuration pair. The total regret decomposes as $R(T) = R_1(T) + R_2(T)$, where $R_1(T)$ captures suboptimality from detector selection and $R_2(T)$ from configuration selection. The combined bound is:*

$$R(T) = R_1(T) + R_2(T) = O(\sqrt{mT \log T} + \sqrt{CKT \log T}) \tag{35}$$

Proof. We combine the regret bounds from Theorems 1 and 2.

From Theorem 1, $R_1(T) = O(\sqrt{mT \log T})$.

For $R_2(T)$, we apply the Cauchy-Schwarz inequality:

$$R_2(T) = \sum_{i=1}^{K} O(\sqrt{C \cdot N_i(T) \log N_i(T)}) \tag{36}$$

$$\leq O\left(\sqrt{C \log T} \sum_{i=1}^{K} \sqrt{N_i(T)}\right) \tag{37}$$

$$\leq O\left(\sqrt{C \log T} \cdot \sqrt{K \sum_{i=1}^{K} N_i(T)}\right) \tag{38}$$

$$= O\left(\sqrt{C \log T} \cdot \sqrt{KT}\right) \tag{39}$$

$$= O(\sqrt{CKT \log T}) \tag{40}$$

where we used $\log N_i(T) \leq \log T$, Cauchy-Schwarz ($\sum_i \sqrt{a_i} \leq \sqrt{K \sum_i a_i}$), and $\sum_i N_i(T) = T$.

Therefore:

$$R(T) = R_1(T) + R_2(T) = O(\sqrt{mT \log T} + \sqrt{CKT \log T}) \tag{41}$$

Since both terms grow as $O(\sqrt{T \log T})$, the average regret vanishes:

$$\lim_{T \to \infty} \frac{R(T)}{T} = \lim_{T \to \infty} O\left(\sqrt{\frac{\log T}{T}}\right) = 0 \tag{42}$$

This guarantees that AutoDD converges to the optimal detector-configuration pair as $T \to \infty$. □

E. Sample Complexity

To achieve average regret at most ϵ, we require $R(T)/T \leq \epsilon$. From the combined regret bound:

$$\frac{R(T)}{T} \leq \frac{c_1\sqrt{mT \log T} + c_2\sqrt{CKT \log T}}{T} = \frac{(c_1\sqrt{m} + c_2\sqrt{CK})\sqrt{\log T}}{\sqrt{T}} \tag{43}$$

for constants c_1, c_2. Setting this equal to ϵ and solving:

$$T = O\left(\frac{(m + CK) \log(K/\delta_{\text{conf}})}{\epsilon^2}\right) \tag{44}$$

With $m = 15$ meta-features, $K = 7$ detectors, and $C = 5$ configurations, achieving $\epsilon = 0.01$ average regret requires approximately $T \approx 5 \times 10^5$ windows, which is reached early in typical streaming applications.

Per-Drift-Type Performance: Table IX shows AutoDD's performance per drift type. AutoDD excels on sudden (88.2%), incremental (80.4%), and recurring (79.6%) drift. Meta-ADD slightly outperforms on gradual drift (82.3% vs. 81.7%) due to its learned drift-type representations.

TABLE IX: Accuracy (%) by drift type (GaussianNB).

Method	Sudden	Gradual	Incremental	Recurring
ADWIN	82.4	71.3	68.5	75.2
DDM	79.8	74.6	72.1	71.8
EDDM	75.2	78.4	74.8	69.3
HDDM-A	81.7	72.1	69.8	74.5
HDDM-W	80.1	76.2	73.5	74.6
Page-Hinkley	81.9	70.8	68.2	75.8
KSWIN	76.5	77.8	75.2	72.1
MDDM	81.3	75.8	74.1	73.9
ADDM	82.9	77.2	75.6	74.8
PUDD	82.1	79.5	76.8	75.4
Meta-ADD	84.6	**82.3**	78.9	77.8
AutoDD	**88.2**	81.7	**80.4**	**79.6**

TABLE X: Detector selection frequency (%) by stream characteristic.

Detector	High Velocity	High Noise	High Dim.
ADWIN	28.4	12.3	15.7
DDM	22.1	8.5	11.2
EDDM	8.3	18.7	14.5
HDDM-A	15.2	10.4	12.8
HDDM-W	12.6	14.2	13.1
Page-Hinkley	5.8	8.1	7.2
KSWIN	7.6	27.8	25.5

Detector Selection Patterns: Table X shows that AutoDD's learned selection policy aligns with domain knowledge: ADWIN and DDM are preferred for high-velocity streams, while KSWIN dominates for high-noise and high-dimensional streams.